



Text Processing Commands

grep, egrep, head, tail ,.....

grep & egrep — find lines that match

Search through text and print only the lines you're looking for.

grep

Looks through a file (or input) line by line and prints every line that contains the pattern you give it.

Think of it as Ctrl+F for the terminal.

```
grep "error" app.log
```

egrep

Same as grep, but understands richer patterns — like matching "cat OR dog" — without extra backslashes.

(It's really just grep -E.)

```
egrep "cat|dog" pets.txt
```

head & tail — peek at a file

Look at just the beginning or end of a file, without opening the whole thing.

head

Shows the first 10 lines of a file by default. Great for a quick look at what a file contains.

```
head -n 5 report.csv
```

tail

Shows the last 10 lines of a file by default. With -f, it keeps watching and shows new lines as they're added — perfect for live logs.

```
tail -f server.log
```

sort & uniq — organize lines

Put lines in order, then remove the duplicates.

sort

Arranges the lines of a file alphabetically or numerically. `uniq` only works properly on sorted input, so these two are used together all the time.

Uppercase is given priority over lowercase

```
sort names.txt
```

uniq

Removes duplicate lines that sit next to each other. Usually used right after `sort`, so all duplicates are next to each other first.

```
sort names.txt | uniq
```

cut & paste — slice and combine columns

Pull out specific columns, or glue lines from files together.

cut

Extracts a specific column or field from each line — like grabbing just one column out of a spreadsheet row.

```
cut -d' ' -f2 people.csv
```

paste

Joins lines from two (or more) files side by side, like laying spreadsheet columns next to each other.

```
paste names.txt scores.txt
```

join & comm — compare and merge files

Match rows between two files, or spot what's shared and different.

join

Combines two sorted files based on a matching field — the command-line version of a database JOIN.

```
join file1.txt file2.txt
```

comm

Compares two sorted files and shows what's unique to each one, and what's common to both, in three neat columns.

```
comm file1.txt file2.txt
```

diff & tr — spot changes, swap characters

See what changed between two files, or transform characters as text streams through.

diff

Compares two files and shows exactly which lines were added, removed, or changed — what powers code review "diffs".

The output includes information about which lines in the files must be changed so that they become identical. If the files match, no output is produced.

```
diff old.txt new.txt
```

tr

Translates or deletes characters as they pass through — e.g. swap every lowercase letter for uppercase.

```
tr 'a-z' 'A-Z' < file.txt
```

nl & awk — number lines, process fields

Add line numbers, or run a full mini-program over each line of text.

nl

Adds line numbers to a file — like head -n but for numbering every line of output instead of limiting it.

```
nl notes.txt
```

awk

A small programming language for text: split each line into fields and print, calculate, or filter them.

```
awk '{print $1}' data.txt
```